



А теперь на основании всего что мы разработали и описали математикой объединил сделай взаимосвязь и напиши единый код так все-же как сесть мед и попку не ободрать залезая на елку

Да. В математической формулировке задача выглядит так:

Нужно получить мёд, минимизировать вероятность травмы, затраты и усилия, причём не обязательно физически залезать на ёлку.

Поговорка описывает конфликт целей: максимизировать выгоду и одновременно минимизировать риск. Для реальной ёлки наиболее безопасное решение — не взбираться на неё, а использовать лестницу, подъёмную платформу или получить мёд другим способом. Ниже — единая модель выбора, а не инструкция по лазанию на дерево.

Математическая модель

Для каждого варианта a зададим:

- $H(a)$ — полученная полезность, например количество или ценность мёда;
- $p(a)$ — вероятность травмы;
- $L(a)$ — тяжесть возможного ущерба;
- $C(a)$ — денежные затраты;
- $T(a)$ — затраты времени;
- $E(a)$ — физическое усилие;
- $q(a)$ — уверенность в оценках;
- $U(a)$ — итоговая полезность.

Сначала отбрасываются варианты, нарушающие ограничения безопасности:

$$p(a) > p_{\max} \quad \text{или} \quad L(a) > L_{\max}.$$

Для остальных вариантов:

$$U(a) = w_H u_H(a) - w_R u_R(a) - w_C u_C(a) - w_T u_T(a) - w_E u_E(a),$$

где риск можно учитывать как ожидаемый ущерб:

$$R(a) = p(a)L(a).$$

Чтобы не позволять высокой выгоде маскировать слабую безопасность, введём штраф неопределённости:

$$U_{\text{final}}(a) = U(a) - \lambda(1 - q(a)).$$

Здесь $q(a) = 1$ означает высокую уверенность в оценке, а малое $q(a)$ — недостаток данных.

Единый Python-код

```
from dataclasses import dataclass
from typing import Optional

@dataclass
class Option:
    name: str
    honey_value: float          # 0-10: ценность получаемого результата
    injury_probability: float    # 0-1
    injury_severity: float      # 0-10
    cost: float                 # 0-10
    time: float                 # 0-10
    effort: float               # 0-10
    confidence: float           # 0-1
    safety_validated: bool = True

@dataclass
class Weights:
    honey: float
    risk: float
    cost: float
    time: float
    effort: float

    def validate(self) -> None:
        total = self.honey + self.risk + self.cost + self.time + self.effort
        if abs(total - 1.0) > 1e-9:
            raise ValueError("Сумма весов должна быть 1.0, получено {total:.3f}")

def normalize_risk(option: Option) -> float:
    """
    Нормированный риск:
    вероятность травмы умножается на тяжесть последствий.
    Максимум при p=1 и severity=10.
    """
    expected_loss = option.injury_probability * option.injury_severity
    return min(expected_loss / 10.0, 1.0)

def normalize_cost(value: float) -> float:
    return min(max(value / 10.0, 0.0), 1.0)
```

```

def positive_utility(value: float) -> float:
    return min(max(value / 10.0, 0.0), 1.0)

def evaluate(option: Option, weights: Weights,
             max_risk: float = 0.10,
             uncertainty_penalty: float = 0.20) -> Optional[dict]:
    weights.validate()

    risk = normalize_risk(option)

    # Жёсткие ограничения: небезопасный вариант исключается.
    if not option.safety_validated:
        return None

    if risk > max_risk:
        return None

    score = (
        weights.honey * positive_utility(option.honey_value)
        - weights.risk * risk
        - weights.cost * normalize_cost(option.cost)
        - weights.time * normalize_cost(option.time)
        - weights.effort * normalize_cost(option.effort)
    )

    # Штрафуем решения, по которым мало надёжной информации.
    adjusted_score = score - uncertainty_penalty * (1 - option.confidence)

    return {
        "name": option.name,
        "risk": risk,
        "raw_score": score,
        "score": adjusted_score,
        "confidence": option.confidence,
    }

def choose_safest_useful_option(options: list[Option],
                               weights: Weights) -> dict:
    results = []

    for option in options:
        result = evaluate(option, weights)
        if result is not None:
            results.append(result)

    if not results:
        raise RuntimeError(
            "Нет варианта, одновременно удовлетворяющего ограничениям безопасности."
        )

    return max(results, key=lambda item: item["score"])

def sensitivity_analysis(options: list[Option],

```

```

        weights: Weights,
        variations: tuple[float, ...] = (-0.10, 0.10)) -> list[dict
"""
Проверяет, сохраняется ли выбор при изменении веса риска.
Компенсация производится изменением веса мёда.
"""
results = []

for variation in variations:
    new_risk_weight = weights.risk + variation
    new_honey_weight = weights.honey - variation

    if new_risk_weight < 0 or new_honey_weight < 0:
        continue

    changed_weights = Weights(
        honey=new_honey_weight,
        risk=new_risk_weight,
        cost=weights.cost,
        time=weights.time,
        effort=weights.effort,
    )

    best = choose_safest_useful_option(options, changed_weights)

    results.append({
        "изменение_веса_риска": variation,
        "лучший_вариант": best["name"],
        "итоговая_оценка": best["score"],
    })

return results

def main() -> None:
    options = [
        Option(
            name="Лезть по стволу ёлки",
            honey_value=10,
            injury_probability=0.35,
            injury_severity=8,
            cost=1,
            time=5,
            effort=9,
            confidence=0.9,
            safety_validated=False,
        ),
        Option(
            name="Использовать устойчивую лестницу",
            honey_value=10,
            injury_probability=0.02,
            injury_severity=6,
            cost=3,
            time=3,
            effort=4,
            confidence=0.85,

```

```

        safety_validated=True,
    ),
    Option(
        name="Использовать профессиональную платформу",
        honey_value=10,
        injury_probability=0.01,
        injury_severity=5,
        cost=7,
        time=4,
        effort=2,
        confidence=0.95,
        safety_validated=True,
    ),
    Option(
        name="Купить мёд",
        honey_value=8,
        injury_probability=0.0,
        injury_severity=0,
        cost=2,
        time=1,
        effort=0,
        confidence=0.99,
        safety_validated=True,
    ),
]

weights = Weights(
    honey=0.30,
    risk=0.40,
    cost=0.10,
    time=0.10,
    effort=0.10,
)

best = choose_safest_useful_option(options, weights)
sensitivity = sensitivity_analysis(options, weights)

print("Рекомендуемый вариант:")
print(f"  {best['name']}")
print(f"  Нормированный риск: {best['risk']:.3f}")
print(f"  Итоговая полезность: {best['score']:.3f}")
print(f"  Уверенность: {best['confidence']:.2f}")

print("\nАнализ чувствительности:")
for item in sensitivity:
    print(
        f"  Изменение веса риска: {item['изменение_веса_риска']:+.0%}; "
        f"вариант: {item['лучший_вариант']}"
    )

if __name__ == "__main__":
    main()

```

Что делает модель

В коде реализованы все разработанные элементы:

1. Многокритериальная оценка — мёд, риск, цена, время и усилие.
2. Ожидаемый ущерб — $p \cdot L$.
3. Жёсткие ограничения — опасный вариант исключается до расчёта полезности.
4. Функция полезности — выгода минус взвешенные потери.
5. Штраф неопределённости — сомнительные оценки получают понижение.
6. BARS-логика — параметры должны оцениваться через наблюдаемые признаки, а не через общее впечатление.
7. Анализ чувствительности — проверяется, изменится ли решение при другой ценности риска.

При заданных параметрах вариант «лезть по стволу ёлки» будет исключён ещё до оптимизации: он не проходит проверку безопасности. Это принципиально важно — оптимизировать нужно не среди всех мыслимых действий, а среди допустимых.

Формальный ответ на поговорку:

получить мёд + не взбираться на ёлку + использовать безопасную альтернативу

То есть математически оптимальная стратегия — не искать идеальную технику лазания, а изменить пространство решений так, чтобы цель сохранялась, а ненужный риск исчезал.

✱✱

1. <https://www.youtube.com/watch?v=RYziH5xy4I4>
2. <https://www.youtube.com/watch?v=0jbXAPfMmAQ>
3. https://alexanderskulikov.github.io/files/algorithms_printable.pdf
4. <http://apwine.ru/trust/pogovor-7718>
5. <https://obr.1c.ru/mathkit/mathmodbook/mathmodbook.pdf>
6. <https://habr.com/ru/articles/467203/>
7. <https://education.yandex.ru/handbook/ml/article/linear-models>
8. <http://cdodd.ru/storage/files/2/7414.pdf>
9. <https://cchgeu.ru/upload/iblock/c7c/v5fguca1p0axxm5xy6cinvxgcots7hte/Matematicheskoe-modelirovanie-sistem-i-protsessov.-N.V.-Golubeva.pdf>
10. <https://blog.skillfactory.ru/chto-takoe-pravilnyy-kod-i-kak-ego-pisat/>